Chapter 9

Advertising and Retail Media Networks

9.1 Introduction: From Organic Ranking to Sponsored Products

In the preceding chapters, you built the core components of a recommender system: a retrieval stage that narrows millions of items to a few hundred candidates (Chapter 5), a ranking stage that scores those candidates using rich features (Chapters 6–7), and a value function layer that translates model outputs into business objectives (Chapter 8). In all of those chapters, the system served a single boss: the user. The goal was to surface the most relevant items for that user, full stop.

This chapter introduces a domain where the system must serve an additional boss: the advertiser. When a retailer sells sponsored product placements alongside organic results, the system is no longer just recommending: it is also running an advertising marketplace. The ML model's predictions now feed directly into an economic mechanism that determines which ads get shown, how much advertisers pay, and ultimately how much revenue the platform earns. Getting the system wrong does not just degrade relevance; it distorts prices, wastes advertiser budgets, and erodes trust in the platform.

This chapter is organized in three parts. First, we will walk through the end-to-end architecture of a Retail Media Network: which is the advertising variant of the multi-stage pipeline you already know and contains more specialized modules. This requires introducing several new concepts (bidding, auctions, pricing) that do not exist in organic recommendation. We will take our time with these because they are the foundation for understanding why the modeling challenges in advertising are different from, and in many ways harder than, those in organic ranking.

Second, we will deep-dive into the Entire Space Multi-Task Model (ESMM), which addresses a subtle but critical problem: if you train a conversion rate model only on users who clicked, the model will be systematically biased when applied to all users at serving time. We will build the model, examine why the bias occurs, and see how ESMM eliminates it.

Third, we will tackle Federated Learning for advertising (FedAds), which addresses a different kind of challenge: what happens when the party that shows the ad and the party that owns the conversion data are different entities/companies that cannot share raw data? We will build a split neural network that trains jointly across two parties without either one seeing the other's features.

Before we begin, let us define the key players in the advertising ecosystem, since these terms will appear throughout the chapter.

9.2 The Advertising Ecosystem: Key Players and Terminology

If you come from organic recommendation, advertising terminology & jargon can feel overwhelming. There are publishers, advertisers, demand-side platforms, supply-side platforms, ad exchanges, and more. And to top it all, stakeholders seem to spend very little time in what you as the Recsys ML engineer think *should* be the most important – retrieval & ranking. Why is that the case? Let us start with the essential roles and build up the intuition from there. from there.

9.2.1 The Core Roles

The Publisher is the entity that owns the surface where ads are displayed. In classical advertising, this is a website or app that has users but wants to monetize their attention. Think of a news website showing banner ads, or a search engine showing sponsored results above organic ones. The publisher controls what the user sees, including the mix of organic content and paid advertisements.

The Advertiser is the entity that wants to promote a product or service. The advertiser pays money to have their ad shown to users who are likely to be interested. An advertiser could be a brand (Nike promoting running shoes), a retailer (Walmart promoting a sale), or a small business (a local bakery promoting on Instagram). The advertiser's primary concern is return on ad spend (ROAS): for every dollar spent on advertising, how much revenue does it generate?

ROAS is worth pausing on because it is the metric that fundamentally shapes how advertising systems are evaluated, and it is subtly different from metrics like CTR or conversion rate that you may be familiar with from Chapters 5–8. CTR and conversion rate are behavioral metrics: they measure the fraction of users who take an action. ROAS, by contrast, is a financial metric. It is defined as (revenue generated from ad-driven purchases) / (total ad spend). A campaign with a high conversion rate can still have a poor ROAS if the products being purchased are low-margin or if the cost-per-click is too high. Conversely, a campaign with a modest conversion rate can have excellent ROAS if even a few conversions yield high-value purchases. This is why the scoring formulas we will see later in this chapter (Section 9.3) combine predicted probabilities with dollar-valued bids: the system needs to reason about economic value, not just user behavior. Throughout this chapter, keep in mind that the ultimate objective is ROAS for the advertiser and revenue for the platform: CTR and CVR are intermediate signals in service of that goal.

The Ad Platform is the infrastructure that connects publishers and advertisers. In large-scale advertising, this involves two specialized systems: a Demand-Side Platform (DSP), which represents advertisers and helps them buy ad placements efficiently, and a Supply-Side Platform (SSP), which represents publishers and helps them sell their ad inventory at the best price. Between them sits an Ad Exchange, which runs the real-time auction that matches ads to available slots. In practice, large companies like Google and Meta operate all three components as a vertically integrated stack.

For the purposes of this chapter, the most important distinction is between two very different advertising architectures: the classical display advertising model, where the publisher and advertiser are separate entities, and the Retail Media Network model, where a single retailer plays all three roles.

9.2.2 Classical Display Advertising vs. Retail Media Networks

In classical display advertising such as banner ads on a news website, clicking an ad redirects the user to the advertiser's own website away from the publisher's website. The publisher never sees what happens after the click. Did the user buy something? Did they bounce immediately? The publisher has no way to know. This creates a fundamental measurement problem: the publisher optimizes ad placement based on clicks (which it can measure), but the advertiser cares about conversions (which only the advertiser can observe). The two parties have misaligned information.

Retail Media Networks (RMNs) eliminate this information asymmetry entirely. An RMN is an advertising platform operated by a retailer: Amazon Sponsored Products, Walmart Connect, Instacart Ads, and Target Roundel are all examples. In an RMN, the retailer is simultaneously the publisher (it shows the ad on its search results page), the ad exchange (it runs the auction), and the transaction platform (the user buys the product on the retailer's own site). Clicking a sponsored product does not redirect the user to a third-party website; it takes them to the product detail page on the retailer's own platform.

This structural difference has profound implications for modeling:

Capability	Classical Display	Retail Media Network
Who sees the conversion?	Only the advertiser	The retailer sees everything
Conversion measurement	Probabilistic (cross-domain tracking, cookies)	Deterministic (retailer owns checkout)
Attribution	Multi-touch, lossy, delayed	Direct, high fidelity
Click-to-purchase funnel	Crosses domain boundaries	Stays on one platform
Feature availability for CVR model	Limited post-click signals	Full user history, cart, purchase data

Because the retailer controls the full funnel from impression to purchase, CVR modeling in RMNs benefits from cleaner labels, richer features, and deterministic attribution. However, the fundamental modeling challenges we will discuss—selection bias in CVR estimation (Section 9.4) and the need for federated learning when parties cannot share data (Section 9.5)—apply to both architectures.

Author's Note: *The RMN business model is economically transformative. Before RMNs, retailers monetized their user traffic only through product margins. Now, advertising revenue has become a major profit center for many retailers. For example, Amazon's advertising business alone generated approximately $69 billion in revenue in 2025 (per Amazon's Q4 2025 earnings report), growing at over 20% year-over-year with high margins. For ML engineers,*

this means that the models you build in this domain have direct, measurable bottomline impact sometimes even more so than the core Search/Recommendations areas.

9.3 The Sponsored Product Pipeline: End to End

Let us now walk through exactly what happens when a user searches for a product on a retailer's site and the system must decide which sponsored products to show alongside the organic results. If you have read Chapters 5 through 8, the first few steps will look familiar with retrieval followed by ranking. But from the scoring step onward, the pipeline diverges significantly from organic recommendation.

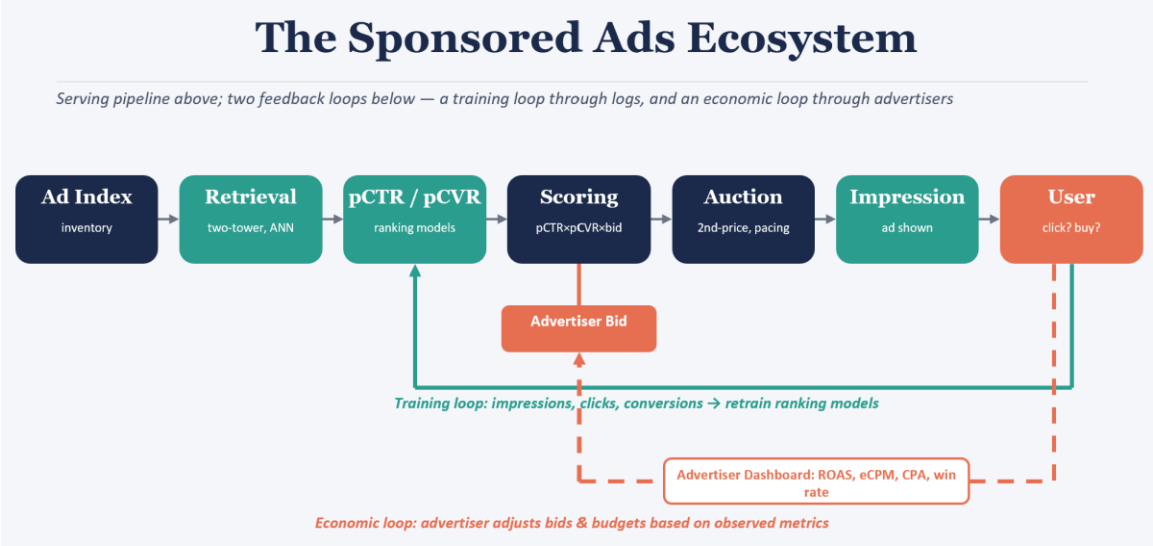


Figure 9.1: The Sponsored Ads Ecosystem. A user request triggers a serving pipeline that runs from the ad index through retrieval, CTR/CVR prediction, scoring, and a second-price auction before an impression is shown. Two feedback loops close over that pipeline: a training loop where logged impressions, clicks, and conversions flow back into retraining the ranking models, and an economic loop where advertisers observe ROAS, eCPM, CPA, and win rate on their dashboards and adjust bids and budgets accordingly. These two loops operate on different timescales and optimize different objectives, which is what makes the advertising ecosystem structurally distinct from organic recommendation.

9.3.1 Retrieval and Feature Enrichment

Lets examine a sponsored search use case: A user searches for "wireless headphones." The retrieval stage kicks in, just as it would for organic results (Chapter 5). But now there are two parallel retrieval paths:

- Organic retrieval: Find the most relevant products from the catalog using the techniques from Chapter 5 (embedding similarity, BM25, collaborative filtering, etc.).

- Sponsored retrieval: Find sponsored products whose advertisers have active campaigns targeting this query. Only products with active bids and remaining budget are eligible.

For each sponsored candidate, the system assembles a feature vector similar to what the ranking model in Chapter 6 would use: query features, user features (browsing history, demographics), and product features (category, price, brand). But there are also features unique to advertising: the advertiser's bid amount, remaining daily budget, and historical campaign performance metrics.

9.3.2 Predicting Clicks and Conversions

The ranking model in organic recommendation (Chapters 6–7) produces a single relevance score per item. In advertising, the system needs two separate predictions:

pCTR – Predicted Click-Through Rate: P(click | impression). What is the probability that the user will click on this sponsored product if we show it? This is conceptually identical to the CTR prediction you would make in an organic ranking model.

pCVR – Predicted Conversion Rate: P(conversion | click). Given that the user clicked, what is the probability that they will subsequently purchase the product? This is a new prediction task that introduces unique modeling challenges that we will explore in depth in Section 9.4.

These two predictions are combined to estimate the expected number of conversions per impression:

```
Expected conversions per impression = pCTR × pCVR
```

This quantity is the bridge between the ML models and the economic layer of the system. It answers the question: if we show this ad to this user, how likely is it to result in a sale?

9.3.3 Scoring: Where ML Meets Economics

Here is where advertising diverges fundamentally from organic recommendation. In organic ranking, the score is a pure relevance signal where higher score means more relevant. In advertising, the score combines relevance with economics of how much the advertiser is willing to pay.

Each advertiser sets a **bid**, which represents the maximum amount they are willing to pay per conversion (in a CPA, or cost-per-acquisition model) or per click (in a CPC, or cost-per-click model). The bid is often not determined by a predictive model instead it is an input from the advertiser's business strategy.

In the most simplistic systems, a ranking score for each sponsored product is computed by multiplying the bid with the model's prediction of expected value:

```
Score = Bid × pCTR × pCVR
```

This formula is the heart of advertising economics. Let us unpack what it means. If an advertiser bids $50 per conversion, and the model predicts a 10% click probability and a 20% conversion-given-click probability, the expected value of showing this ad is $50 × 0.10 × 0.20 = $1.00 per impression. This represents the expected revenue the platform would earn from this ad slot.

The sponsored products are then sorted by this score in descending order. The highest-scoring ads fill the available sponsored slots. This step of deciding which ads to show is called allocation.

Notice something important: the allocation decision depends on both the advertiser's willingness to pay (the bid) and the model's prediction of relevance (pCTR × pCVR). A high bid alone is not enough to win because if the model predicts the user is unlikely to click, the score for that item will be low despite a high bid. This design encourages advertisers to promote relevant products rather than simply outbidding competitors with irrelevant ads.

Note that in some RMN systems, the bid price that is used in the multiplicative formula is itself the output of a heuristic/model rather than the original bid prices set by the advertisers. This allows for a dynamic bid price depending on how much budget remains , etc. For example, an ad that is doing really well can be 'throttled' by the bidding model by lowering the bid price for subsequent score calculations. Thereby temporarily lowering the chances of that ad item coming up as the top rank.

9.3.4 The Auction: How Pricing Works

Once the system has decided which ads to show (allocation), it must determine how much each winning advertiser pays. This is the auction and it is often the most confusing part of the advertising pipeline for ML engineers encountering it for the first time.

The key insight is that allocation and pricing are two separate steps. Allocation (who gets shown) is just sorting by score, as described above. Pricing (how much they pay) is determined by a separate rule, typically a variant of the second-price auction.

In a second-price auction, the winner does not pay their own bid. They pay the minimum amount that would have been necessary to still win, which is determined by the next-highest competitor's score. The formula, in a CPA model, is:

```
CPA_paid = Score_of_runner_up / (pCTR_winner × pCVR_winner)
```

Why not simply charge the winner their own bid (i.e., a first-price auction)? Because in a first-price auction, advertisers have an incentive to shade their bids—bidding less than their true value to avoid overpaying. This leads to unstable auction dynamics where everyone is trying to guess the minimum bid needed to win. The second-price mechanism removes this incentive: the optimal strategy is to bid your true value, because you will never pay more than the next competitor requires. This is a foundational result in auction theory, and it is why most major ad platforms use second-price (or Generalized Second Price) variants.

Let us walk through a concrete example to solidify this.

Toy Example: Assume that a customer request is sent to the publisher's search page where three fictional advertisers compete for one sponsored slot on the query "running shoes". Assume each of them has 1 SKU. Assume that the Bid price given below is what is set by the advertisers in the RMN portal. For each of the particular item, assume that the pCTR and pCVR models predict the corresponding click and conversion probabilities respectively. Then the score is a simple multiplication as shown in the last column

Table 1: Toy example with 3 advertisers, their bids, predicted click and conversion rates leading to the score calculation.

Advertiser	CPA Bid	pCTR	pCVR	Score = Bid × pCTR × pCVR
A (Mike)	$50	0.10	0.20	$1.00
B (Edidas)	$80	0.05	0.10	$0.40
C (Mew Balance)	$40	0.20	0.05	$0.40

The Advertiser 'Mike' wins with a score of $1.00 despite not having the highest bid price. On the other hand, despite Edidas bidding $80 (the highest bid), in this fictional example, the RMN models predicted that the particular Edidas SKU had too low of a predicted click and conversion rates. So their expected value per impression is only $0.40. Mew Balance has the highest pCTR (users are likely to click) but very low pCVR (they rarely convert), which also yields $0.40.

Now lets turn to the question of how much does Mike pay the advertising platform after the scoring step: Mike's price is determined by the runner-up's score, ***not*** its own bid:

```
CPA_paid = 0.40 / (0.10 × 0.20) = 0.40 / 0.02 = $20
```

Mike bid $50 per conversion but only pays $20. The $30 gap is the "surplus" that Nike retains: their effective ROAS is better than their bid implied. This is exactly how second-price auctions are supposed to work: the winner pays just enough to beat the competition, not their maximum willingness to pay.

9.3.5 Blending Sponsored and Organic Results

After the auction determines the winning sponsored products and their prices, the final step is blending: interleaving sponsored results with organic results on the search results page. This is a policy decision, not a pure ML decision. Typical rules include:

- **Reserved Slots**: Reserve specific positions for sponsored results (e.g., slots 1 and 4 on the first page).
- **Maximum Ads**: Cap the total number of sponsored results per page (e.g., no more than 3 out of 20 displayed).

- **Minimum Relevance**: Require a minimum relevance threshold: even if an ad wins the auction, it may be suppressed if the CTR and/or CVR models predict it is highly irrelevant to the user (based on the pCTR and pCVR)

The blending policy reflects a fundamental tension: showing more ads generates more ad revenue in the short term, but if ads degrade the user experience, users will engage less with the platform over time, reducing both organic and ad-driven revenue. Managing this tension is an ongoing challenge for RMN teams who are often organizationally distinct (with separate ML/Engineering teams) from the rest of the ecommerce , search and recommendations teams.

9.3.6 Impression Logging

At the moment an ad is shown, the system logs a detailed record of the decision. A few useful pieces of metadata are as follows:

- The predicted pCTR and pCVR for the shown ad
- The clearing price (what the advertiser will pay if clicked)
- The runner-up's score (needed for price computation)
- The slot position where the ad was placed
- The advertiser's remaining budget and any pacing multipliers applied

These logs serve multiple purposes: billing (charging the advertiser the correct amount when a click or conversion occurs), model retraining (the next version of the CTR/CVR models trains on this data), and auditability (the advertiser can verify they were charged correctly).

9.3.7 Why Calibration Is Critical in Advertising

This section addresses a subtle but deeply important difference between organic ranking and advertising. It is worth spending time on because it changes how you should think about model quality in this domain as opposed to assuming that ranking metrics are sufficient.

In 'organic' ranking (Chapters 6–7), the main thing that matters is the relative ordering of predictions. A model that assigns scores of 0.9, 0.5, 0.1 to three items works exactly as well as one that assigns 0.09, 0.05, 0.01, both produce the same ranking. The standard evaluation metric, AUC, captures exactly this: the ability to rank positives above negatives, regardless of the absolute score values.

In advertising, getting the relative ranking correct is no longer sufficient due to the multiplicative effects when converting from the unitless quantities of pCTR, pCVR into the monetary space:

```
Score = Bid × pCTR × pCVR
```

The absolute values of pCTR and pCVR are multiplied with real dollar amounts. You might ask: if the CVR model systematically predicts probabilities that are 2× too high for every ad,

wouldn't all scores double uniformly, leaving the relative rankings unchanged? In that idealized scenario, yes—the same ads would win the same auctions. But calibration still matters for several reasons, even in that perfectly uniform case:

1. **Budget pacing breaks down.** Most advertisers set daily or campaign-level budgets. The platform's budget pacing system uses the predicted conversion probability to estimate how quickly an advertiser's budget will be consumed. If pCVR is 2× too high, the pacer thinks each impression is twice as valuable, so it spends the budget more aggressively. The advertiser exhausts their daily budget by midday instead of spreading it across the full day, missing high-value evening traffic entirely.
2. **Second-price payments are inflated.** In a second-price auction, the winner pays slightly above the second-highest bid. Even if the rankings don't change, the absolute second-price is derived from the second-place advertiser's score, which is also 2× inflated. The winning advertiser pays roughly twice what they should for the same impression. When they measure actual conversions, their observed cost-per-acquisition (CPA) is far higher than expected. Their ROAS collapses.
3. **Cross-channel budget allocation shifts.** Large advertisers allocate spend across multiple platforms (Google, Meta, Amazon, Walmart, etc.) based on each platform's reported predicted conversion rates. If one platform's pCVR is systematically inflated, the advertiser initially over-allocates budget to that platform. But when actual conversions disappoint, they pull spend back, often over-correcting, and redirect it to competing platforms. The trust damage is lasting.

And in practice, miscalibration is never perfectly uniform. Different ad categories, user segments, and contexts are miscalibrated by different amounts. A model that over-predicts CVR by 3× for electronics but only 1.2× for apparel will systematically shift auction wins toward electronics advertisers, distorting both which ads users see and how efficiently the marketplace allocates spend. The net effect is the same whether uniform or non-uniform: advertisers lose trust, the platform loses long-term revenue, and users see less relevant ads.

Conversely, if pCVR is systematically too low, advertisers under-bid implicitly. They lose auctions they should win, the platform leaves revenue on the table, and the overall marketplace is less efficient.

This is why model calibration is important in advertising: when the model predicts a 5% probability, approximately 5% of those events must occur across a reasonable sample. A model can have excellent PR-AUC (it correctly ranks higher-probability events above lower-probability ones) and still be poorly calibrated (the actual probabilities are systematically off). In organic recommendation, this does not matter. In advertising, it directly distorts prices and budget allocation.

In the accompanying tutorials, we implement three standard post-hoc calibration techniques:

Platt Scaling: Fits a logistic regression on the validation set's predicted probabilities to learn a mapping from uncalibrated to calibrated probabilities. Simple and effective when the miscalibration is approximately monotonic.

Isotonic Regression: A non-parametric approach that learns a monotonic step function mapping predictions to calibrated probabilities. More flexible than Platt scaling but requires more validation data.

Temperature Scaling: Divides model logits by a single learned scalar T before applying the sigmoid. Conceptually the simplest approach, specific to neural network outputs.

We measure calibration quality using Expected Calibration Error (ECE), which bins predictions into groups and measures the average gap between predicted probability and observed frequency within each bin. Lower ECE means better calibration. The code for all of this is in utils/calibration.py.

9.3.8 The Auction-Model Feedback Loop

There is one more structural difference between advertising and organic recommendation that is worth understanding before we move into modeling. In organic ranking, the model influences what users see, and user behavior generates training data for the next model. This creates a well-known feedback loop (position bias, as discussed in Chapter 7).

In advertising, the feedback loop is tighter and has an additional economic dimension:

1. The model predicts pCTR and pCVR on the eligible sponsored items.
2. These predictions determine which ads win the auction (allocation).
3. The auction determines what advertisers pay.
4. Advertisers observe their actual ROI ("Is the effective CPA close to their target?").
5. Advertisers adjust their bids based on observed ROI.
6. Changed bids alter which ads win future auctions.
7. Changed allocation changes the distribution of training data for the next model.
8. The CTR/CVR models retrain on this new data distribution.

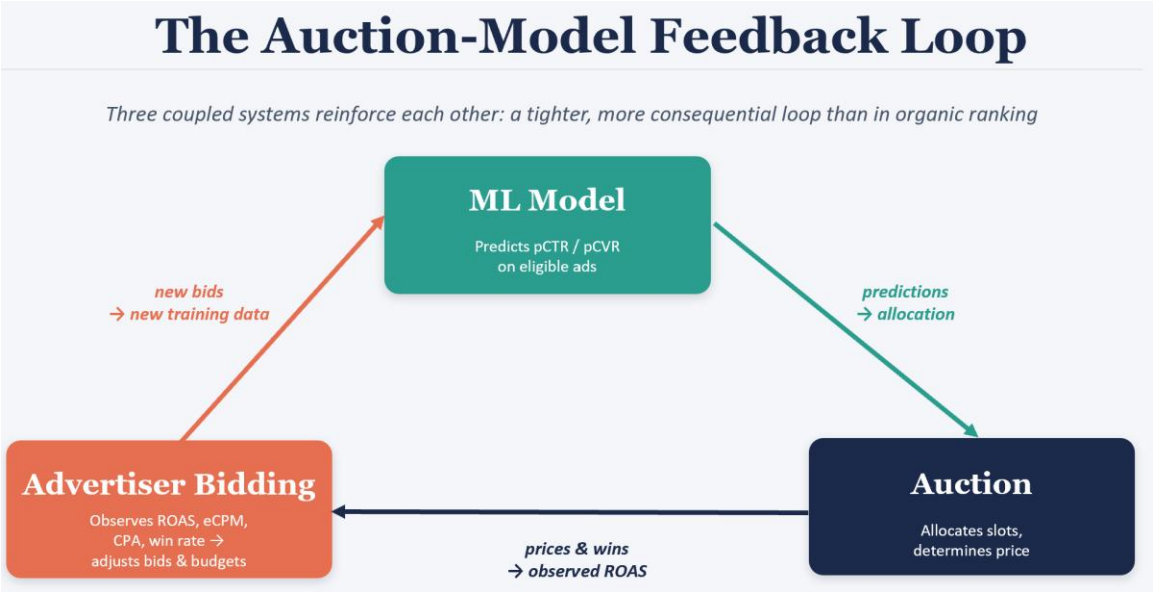


Figure 9.2: The Auction-Model Feedback Loop. Unlike organic recommendation, where the feedback loop involves only the model and user behavior, advertising introduces a third actor, the advertiser's bidding strategy, creating a tightly coupled three-way system where model bias, auction outcomes, and bid adjustments reinforce each other.

The key difference from organic ranking is that there are now three coupled systems: the ML model, the auction mechanism, and the advertiser's bidding strategy. A biased model creates biased auction outcomes, which creates biased training data, which reinforces the bias. And because the feedback loop is often shorter compared with organic recommendations, the consequences are more noticeable and more urgent to be resolved if the system is volatile.

It is worth pausing on step 4, because the advertiser's observation surface is much richer than a single ROI number. Sophisticated advertisers watch a whole dashboard: ROAS against a target, effective CPA, eCPM to compare inventory quality across placements and publishers, win rate and impression share to check whether their bids are even competitive, and CTR and CVR broken down by funnel stage to diagnose where a campaign is leaking. The more mature ones also run holdout experiments to estimate incrementality, the causal lift of ads over an unadvertised baseline, because a high ROAS on users who would have purchased anyway is not actually generating value.

Similarly, "adjust their bids" understates the lever set that sophisticated advertisers pull on. In practice they also adjust daily and campaign budgets, bid modifiers along dimensions like device, geography, daypart and audience segment, targeting rules and negative keywords, and creative rotations. Each of these levers changes what the auction sees, which changes what the ranker observes, which closes the loop. The practical implication for ML engineers is that the data distribution your model trains on tomorrow reflects not just yesterday's model outputs but the aggregated reaction of every advertiser's optimization system to

those outputs, a system that updates on timescales ranging from minutes (automated bidders) to days (manual campaign managers).

With this conceptual foundation in place, we are now ready to dive into two amongst the many modeling challenges that make advertising uniquely difficult: selection bias in CVR estimation (Section 9.4), and privacy-preserving federated learning across parties that cannot share data (Section 9.5).

9.4 Conversion Rate Modeling: The Selection Bias Problem and ESMM

Of the two predictions the advertising system needs (pCTR and pCVR), the CTR model is straightforward if you are coming from Search ranking or Recsys space. You train it on all impressions, with click/no-click as the label. This is essentially the same ranking model you built in Chapter 6, just with a binary click label instead of a relevance label.

The CVR model is where things get tricky. And the subtlety is not in the model architecture, it is in what constitutes the training data.

9.4.1 The Problem: Training on Clicked Samples Only

The conversion rate, formally represented as P(conversion | click), is the probability that a user who clicked on an ad will subsequently purchase the product. The standard approach is to train this model on clicked samples only:

```
Training data:  {(features, label) | user clicked on the ad}
                label = 1 if the user purchased, 0 otherwise

At inference:   Applied to ALL impressions (clicked and non-clicked)
                because the auction score needs pCTR × pCVR for every ad
```

This seems reasonable at first glance. After all, conversion is defined as a post-click event, so it makes sense to only train on data where a click occurred. But there is a subtle and important problem: the distribution of features among users who clicked is different from the distribution of features among all users.

Think about it: users who click on ads tend to be more engaged, more likely to be in a buying mood, and more likely to come from certain demographics or device types. When you train a CVR model exclusively on clicked samples, the model learns the conversion patterns of this biased subset. But at inference time, the model is applied to all users—including the vast majority who would not have clicked. The model has never seen what "non-clicker" features look like, so it generalizes poorly to them. The predicted CVR will likely be an over-estimate if we naively use the model directly.

This distribution mismatch is called sample selection bias (SSB), and it is analogous to a medical study that only enrolls hospital patients and then applies its findings to the general population.

At this point, you might wonder: why not sidestep the problem entirely by modeling P(conversion | impression) directly, training on all impressions instead of just clicks? After all, the conversion label (did the user eventually buy?) is observable for every impression, not just for clicked ones. The reason is that this would break the scoring formula used in the auction. Recall from Section 9.3 that the auction score is:

$$Score = Bid \times pCTR \times pCVR$$

This formula assumes pCVR means P(conversion | click)—the probability of conversion given that the user has already clicked. The product pCTR × pCVR then decomposes the full funnel probability as P(click) × P(conversion | click) = P(click AND conversion). If you instead modeled P(conversion | impression) directly and plugged it into this formula, you would be computing Bid × pCTR × P(conversion | impression), which double-counts the click probability. The resulting score would no longer represent the expected value of the impression, and the auction's economic guarantees would break down.

What we actually want is to model P(click AND conversion | impression) over the full impression space, which avoids the selection bias, while still preserving the decomposition into separate pCTR and pCVR predictions, since the platform needs both signals independently (pCTR for click-based CPC billing, pCVR for conversion-based CPA billing, and the two combined differently depending on the advertiser's campaign type). This is precisely the insight behind the Entire Space Multi-Task Model (ESMM), which we will develop in the next section. ESMM threads this needle by maintaining two separate towers, one for pCTR, one for pCVR, but training them jointly through the identity CTCVR = pCTR × pCVR, with the CTCVR loss computed over all impressions. The CVR tower learns P(conversion | click) without ever being exposed to the selection bias, because its gradients flow through the multiplication with pCTR across the full impression space.

9.4.2 An Intuitive Example

To make the bias concrete, consider a simplified fictional scenario. Assume a retailer with a RMN platform has built CTR, CVR models with just 1 feature for a sponsored search experience: the user's mobile operating system (iOS or Android). Assume that clicking on the sponsored item redirects them internally to the item's product page where the users can add-to-cart and move further down the funnel to the ultimate conversion. Now, say that due to a rendering bug, sponsored search ads display incorrectly on Android devices causing 0 users from Android to click on it. The clickstream logs however record all the impressions. While the bug is present, 100% of sponsored clicks on the search page come exclusively from iOS users. Assume however that after clicking and landing on the product page, iOS and Android users convert at the same rate, say 10%.

A CVR model trained on clicked samples only, would learn from training data containing exclusively iOS users. It has never seen an Android user in its training data. When applied to all impressions at serving time, it has no basis for predicting CVR for Android users.

Now, you might think: "Why not just add the non-clicked impressions as negative conversion examples?" But this proposal to incorporate Android user label as 0 is incorrect, and understanding why is the key to this entire section.

Android users did not click. But the reason they did not click is the UI/UX rendering bug, not because they were uninterested in the product. If you label all non-clicked impressions as "did not convert," you are conflating two very different things: (1) users who saw the ad, were interested, but could not click due to a bug, and (2) users who genuinely were not interested. The model would learn that Android users never convert, which is factually wrong: it's that they never get the chance to click and enter the conversion funnel.

The rendering bug is an extreme example just to illustrate the concept, but the same phenomenon occurs in subtle ways in real systems. Certain ad placements get lower visibility, certain user segments are less inclined to click on sponsored labels, and certain product categories attract window-shoppers who browse but rarely click. All of these create systematic differences between clickers and non-clickers that have nothing to do with conversion propensity.

9.4.3 A Second Problem: Data Sparsity

Selection bias is compounded by extreme data sparsity. Consider the typical conversion funnel in advertising (using the Alibaba ads dataset):

Funnel Stage	Approximate Rate	Relative to Impressions
Impressions	100%	42.3M (Ali-CCP dataset)
Clicks	~4–5%	~2M
Conversions (purchases)	~2% of clicks	~40K

If you train CVR on clicked samples only, your training set is 2M samples with only 40K positives, a 2% positive rate. This makes learning the conversion signal extremely difficult, especially for tail queries or new products with very few clicks.

The ESMM model addresses both problems simultaneously.

9.4.4 The ESMM Solution: Entire-Space Training

The Entire Space Multi-Task Model (Ma et al., SIGIR 2018) takes a clever approach. Instead of training CVR directly on clicked samples, it exploits the sequential structure of the advertising funnel:

```
Impression → Click → Conversion
```

ESMM models two quantities, both trained on the entire impression space (all 42.3M samples, not just the 2M clicked ones):

- CTR = P(click | impression) - supervised predictions on click labels on all impressions.

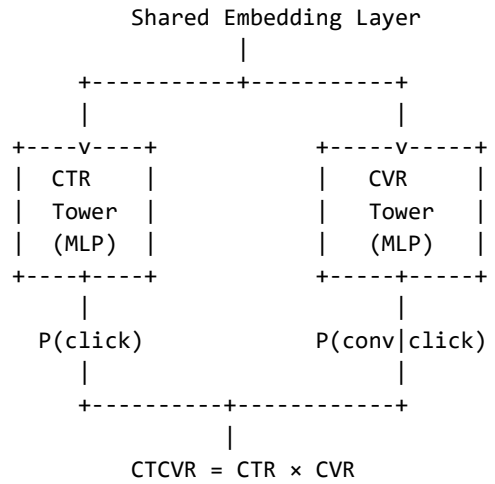
- CTCVR = P(click AND conversion | impression) - supervised predictions on purchase labels on all impressions. Sample is positive only if the user both clicked **and** purchased.

The CVR is never directly supervised. Instead, it is derived through the multiplicative constraint:

```
CTCVR = CTR × CVR

Therefore:
CVR = CTCVR / CTR = P(conversion | click)
```

The neural architecture presented in the original paper consists of two parallel MLP towers sharing a single embedding layer. One tower predicts CTR, the other predicts CVR. Their outputs are multiplied to produce CTCVR:



The training loss has two terms, both computed on all impressions:

- `loss_ctr   = BCE(P(click), click_label)          # All samples have click labels`
- `loss_ctcvr = BCE(CTR × CVR, purchase_label)      # All samples have purchase labels`

```
total_loss = loss_ctr + loss_ctcvr
```

There is no direct CVR loss. The CVR tower receives gradients only indirectly, through the CTCVR loss. This is the key insight: by training CTCVR on the entire impression space, the CVR tower learns from all the samples, not just the clicked ones. The selection bias is eliminated because the training distribution now matches the inference distribution: both cover the entire impression space.

The shared embedding layer is important for the sparsity problem: the CTR tower, which has abundant training signal, learns strong feature representations that are shared with the CVR tower. This is a form of transfer learning within the model.

9.4.5 Critical Assumption: The Sequential Funnel

ESMM relies on the factorization CTCVR = CTR × CVR, which assumes that conversions can only happen after a click. The user experience flow must be strictly sequential: see the ad → click the ad → purchase the product. This means CTCVR is the joint probability of click and conversion, which can be decomposed as the product of the marginal click probability and the conditional conversion probability.

This assumption holds for:

- Traditional display ads where the click takes the user to a product page
- Search ads with landing pages
- Any funnel where a click is a prerequisite for subsequent conversion

This assumption breaks down for:

- View-through conversions: The user sees an ad but does not click. Later, they navigate to the product independently and purchase. The conversion happened without a click.
- One-click purchases: Platforms that offer "Buy Now" buttons that bypass the traditional click-to-product-page-to-purchase funnel.
- Multi-touch attribution: The conversion is attributed to multiple ad exposures across different channels, not a single click event.

For systems where conversions can happen without a click, ESMM's factorization is invalid. Alternative approaches include modeling P(conversion | impression) directly, or using a multi-path architecture with separate branches for click-through and view-through conversions. The Ali-CCP dataset we use follows the traditional sequential funnel, so ESMM is appropriate.

9.4.6 Code Walkthrough

The ESMM model is implemented in models/esmm.py. Let us walk through the key components.

Shared Embedding Layer. All sparse features (user demographics, item category, brand, behavioral cross-features) are hashed to a fixed vocabulary of 100K buckets and embedded into a 16-dimensional space. A padding index of 0 is reserved for empty feature slots:

```
self.embedding = nn.Embedding(
    num_embeddings=config.vocab_size + 1,  # +1 for padding
    embedding_dim=config.embed_dim,         # 16
    padding_idx=0,
)
```

Feature Pooling. Each sample has up to 50 sparse features. These are embedded individually and then mean-pooled (with masking to ignore padding positions) to produce a single fixed-size vector per sample:

```
mask = (feature_ids != 0).float().unsqueeze(-1)       # Identify real features vs
padding
masked_embeddings = embeddings * mask                  # Zero out padding embeddings
pooled = masked_embeddings.sum(dim=1) / mask.sum(dim=1).clamp(min=1)  # Mean pool
```

Parallel Towers. Both CTR and CVR towers are identical MLPs (256 → 128 → 64 hidden units, with batch normalization and dropout), each producing a single sigmoid probability:

```
ctr_logit = self.ctr_tower(pooled)
ctr = torch.sigmoid(ctr_logit)

cvr_logit = self.cvr_tower(pooled)
cvr = torch.sigmoid(cvr_logit)
```

Multiplicative Combination. The CTCVR output is the product of CTR and CVR, with clamping to prevent numerical underflow when both probabilities are very small:

```
ctr_clamped = torch.clamp(ctr, min=1e-7, max=1-1e-7)
cvr_clamped = torch.clamp(cvr, min=1e-7, max=1-1e-7)
ctcvr = ctr_clamped * cvr_clamped
```

Loss Computation. Two loss terms, both on all samples. No direct CVR loss:

```
loss_ctr   = BCE(ctr, click_labels)         # Direct CTR supervision on all samples
loss_ctcvr = BCE(ctcvr, purchase_labels)    # CTCVR supervision on all samples
total_loss = loss_ctr + loss_ctcvr          # CVR tower learns only through CTCVR path
```

9.4.7 The Gradient Asymmetry Problem

There is a subtle issue with ESMM that the original paper does not deeply discuss but that practitioners should be aware of: the CTR and CVR towers receive very different gradient magnitudes.

The CTR tower receives gradients from two sources: strong, direct supervision from loss_ctr (computed on all samples with abundant click labels), and a smaller contribution from loss_ctcvr scaled by p_cvr.

The CVR tower receives gradients from only one source: loss_ctcvr, scaled by p_ctr. Since p_ctr is typically small (around 5%), the effective gradient reaching the CVR tower is roughly 20× weaker than the CTR tower's gradient. This means the CVR tower learns much more slowly.

In practice, the shared embedding layer partially compensates because the CVR tower benefits from embeddings that are being rapidly improved by the CTR tower's strong gradients. But if during your model training, you observe slow convergence of CVR metrics, the gradient asymmetry is a likely cause. The accompanying code includes an experimental option (not from the original paper) to add an auxiliary CVR loss on clicked samples, which gives the CVR tower a direct gradient signal at the cost of reintroducing a small amount of selection bias.

9.4.8 Handling Class Imbalance with Focal Loss

Advertising data has extreme class imbalance: only ~5% of impressions result in clicks, and only ~2% of clicks result in conversions. Standard binary cross-entropy loss is dominated by the vast majority of easy negative examples (the 95% of impressions where no click occurred), leaving little learning signal for the rare positive examples.

Focal loss (Lin et al., ICCV 2017) addresses this by down-weighting well-classified examples and focusing learning capacity on the hard, informative cases:

```
FL(p_t) = -α × (1 - p_t)^γ × log(p_t)
```

When the model correctly classifies an easy negative (p_t near 1.0), the (1 - p_t)^γ term is near zero, effectively removing that sample from the loss. When the model is wrong about a hard positive (p_t near 0), the full loss is retained. With γ = 2 (the default), a correctly classified sample with 95% confidence contributes 400× less to the loss than a misclassified sample. This is implemented in models/esmm.py and is enabled by default for both the CTR and CTCVR loss terms.

9.4.9 Baseline Comparison

To demonstrate the value of entire-space training, the training script (train_esmm.py) trains both ESMM and a CVRBaseline model that trains only on clicked samples. The baseline exhibits exactly the problems the theory predicts: it achieves lower AUC on both CVR and CTCVR tasks, and its predicted conversion probabilities are poorly calibrated, systematically overestimating the conversion rate. This is the direct consequence of selection bias: the model has only seen engaged, clicker behavior during training, and overgeneralizes this to all users at inference time.

9.4.10 Dataset: Ali-CCP

We use the Alibaba Click and Conversion Prediction (Ali-CCP) dataset for the ESMM experiments. This dataset is ideal because it includes both clicked and non-clicked impressions, most public advertising datasets only contain clicked samples, which would make entire-space training impossible.

Attribute	Value
Total impressions	42.3M
Unique users	730K
CTR	~4–5%
CVR (post-click)	~2%
Features	Sparse (user demographics, item category, brand, behavioral crosses)

A preprocessing script (scripts/preprocess_ali_ccp.py) handles the dataset's unusual binary-delimited format and converts it to clean parquet files. For 16GB laptops, we recommend processing a 5% sample (~2.1M impressions). The full dataset requires 32GB+ RAM and is a good homework exercise for running on cloud compute.

9.5 Federated Learning for Advertising

(This is a highly specialized topic. Readers can skip this section entirely without loss of continuity with the rest of the book)

In the previous section, we assumed that the entity training the CVR model has access to all the data: user features, item features, click labels, and conversion labels. This is true in a Retail Media Network, where the retailer controls the full funnel. But in many real-world advertising scenarios, the data is split across organizations.

9.5.1 The Problem: Siloed Data Across Organizations

Consider the advertising relationship between a news website (the publisher) and a shoe brand (the advertiser). The publisher sees what users do before clicking: which articles they read, what time they browse, which device they use, and whether they click on ads. The advertiser sees what happens after the click: did the user land on the product page, did they add to cart, did they purchase?

Neither party has the full picture:

Party	What They Know	What They Don't Know
Publisher	User browsing behavior, page context, click/no-click	Whether the user purchased after clicking
Advertiser	Post-click behavior, purchase history, conversion labels	What the user was doing before they clicked

If they could combine their data, the resulting CVR model would be significantly more powerful—the publisher's pre-click signals would help predict who is likely to convert, while the advertiser's conversion labels would provide the ground truth. But sharing raw data is infeasible due to privacy regulations (GDPR, CCPA), competitive concerns (neither party wants to reveal their user base or behavioral data), and user trust.

This is the problem that Vertical Federated Learning (VFL) addresses.

9.5.2 Vertical vs. Horizontal Federated Learning

You may have heard of "federated learning" in the context of mobile keyboards or healthcare, those are typically horizontal federated learning, where multiple parties have the same features for different users (e.g., multiple hospitals each have patient records with the same schema, but for different patients).

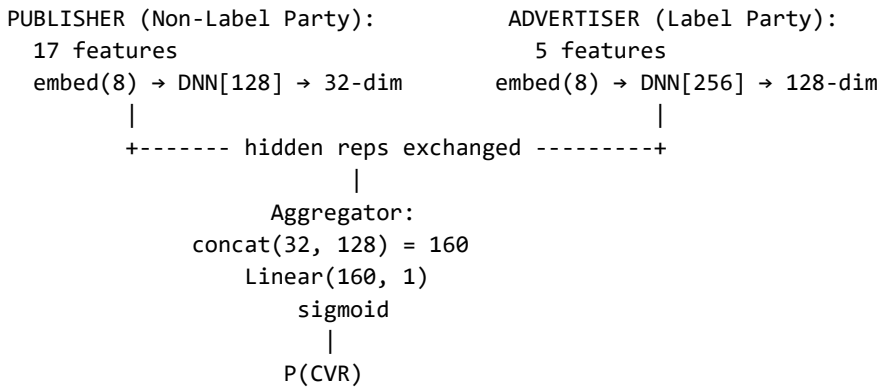
Vertical federated learning is a different setting: the parties have different features for the same users. In our advertising example, the publisher and advertiser both see the same users (those who clicked an ad and were redirected), but each party has its own set of features about those users. The features are partitioned vertically across columns, not horizontally across rows.

9.5.3 The Solution: Split Neural Networks

The core idea of VFL for advertising is to split the neural network at a boundary that aligns with the data partition:

1. Each party trains its own "bottom" model on their local features, producing a compact hidden representation.
2. Only the hidden representations are exchanged, not the raw features.
3. An aggregator (hosted by the party that owns the labels) concatenates the hidden representations and produces the final prediction.
4. Gradients flow back through the same path: the aggregator computes the loss, backpropagates through the aggregator layer, and sends the relevant gradients back to each party for local backpropagation.

We follow the architecture from the FedAds paper (Wei et al., SIGIR 2023). In their setup, the publisher's bottom tower takes 17 features through an embedding layer and a DNN, producing a 32-dimensional hidden vector. The advertiser's bottom tower takes 5 features through its own embedding and DNN, producing a 128-dimensional hidden vector. The aggregator concatenates these (32 + 128 = 160 dimensions), passes through a single linear layer, and applies a sigmoid to output P(CVR):



The hidden representation dimensions (32 and 128) and network architectures are specific to the FedAds paper's design choices and the relative feature richness of each party. In a different application, these would be tuned as hyperparameters based on the number and informativeness of each party's features.

The critical privacy property: raw features never leave their party. The publisher does not see the advertiser's conversion data, and the advertiser does not see the publisher's browsing history. Only the compact hidden representations (32-dim and 128-dim vectors) and corresponding gradients cross the boundary.

9.5.4 Before Training: Private Set Intersection

Before VFL training can begin, the two parties must identify which users appear in both their datasets. The publisher has browsing logs for millions of users; the advertiser has conversion logs for a different (partially overlapping) set of users. They need to find the

intersection ie users present on both sides, without revealing users that exist only on one side.

This requires Private Set Intersection (PSI), a cryptographic protocol that computes the intersection of two ID sets without revealing any non-intersecting elements:

```
Publisher IDs:  {Alice, Bob, Charlie}
Advertiser IDs: {Bob, Charlie, Diana}

PSI output:     {Bob, Charlie}    ← "aligned" samples
                                     Alice and Diana remain hidden from the other party
```

Only aligned samples where the users present on both sides, can be used in VFL training, because you need features from both parties plus the label. In real ad ecosystems, the match rate can be as low as 10–30% due to cookie restrictions, iOS IDFA deprecation, and walled garden effects. This limited overlap is a practical upper bound constraint on VFL's effectiveness (as opposed to the theoretical effectiveness of VFL).

9.5.5 The FedAds Dataset

We use the FedAds dataset (Wei et al., SIGIR 2023), a real advertising dataset from Alibaba with pre-partitioned features explicitly labeled by party:

Attribute	Value
Aligned samples	2.6M (both parties have features)
Unaligned samples	10.4M (publisher features only)
CVR	~0.55%
Publisher features (l_* prefix)	17 (item features, user browsing, click context)
Advertiser features (f_* prefix)	5 (user purchase patterns, cross-features)

The 10.4M unaligned samples represent impressions where the publisher had user data but the advertiser had no matching record, likely because the user had never visited the advertiser's platform. In the basic VanillaVFL approach we implement, these are discarded. More advanced techniques (like the FedCVR approach in the paper) try to leverage unaligned samples through pseudo-labeling or representation pre-training; this is beyond the scope of this chapter, but we have guidance on this as a homework exercise.

Important note on feature counts: In the paper's internal Alibaba system, the advertiser (label party) had 16 features and the publisher had 7. In the public dataset, this ratio is essentially flipped: the publisher has 17 features and the advertiser has only 5. This means the publisher's tower is much more powerful in the public dataset than in the paper's internal experiments, and absolute AUC numbers will differ from the paper's Table 3. The relative ordering of methods (VFL improves over single-party models) should hold, but readers should be aware of this discrepancy when comparing results with the original paper.

9.5.6 Code Walkthrough

The split neural network is implemented in models/split_nn.py. Four models are provided for comparison:

SplitNN (VanillaVFL): The federated model where both parties contribute. This is what you would deploy in a real cross-party collaboration:

```
class SplitNN(nn.Module):
    def __init__(self, config):
        self.local_tower = LocalTower(config)   # Publisher tower
        self.fed_tower = FederatedTower(config)  # Advertiser tower
        self.aggregator = Aggregator(config)      # Combines both

    def forward(self, local_ids, fed_ids):
        h_local = self.local_tower(local_ids)    # Publisher hidden rep
        h_fed = self.fed_tower(fed_ids)           # Advertiser hidden rep
        cvr = self.aggregator(h_local, h_fed)     # Joint prediction
        return {'cvr': cvr, 'h_local': h_local, 'h_fed': h_fed}
```

CentralizedBaseline (ORALE): Both parties share all raw features and a model built on this will give you the theoretical upper bound representing what you could achieve if privacy were not a concern. This tells you the "price of privacy": how much prediction quality is lost by using VFL instead of combining all data.

LabelPartyOnlyModel: Uses only the advertiser's 5 features. This represents what the advertiser could achieve on their own, without any collaboration with the Publisher. If VFL cannot significantly beat this baseline, the operational complexity of setting up cross-party training is not justified.

NonLabelPartyOnlyModel: Uses only the publisher's 17 features. Not in the original paper, but useful for understanding how much predictive power comes from each party's features.

9.5.7 Training Protocol

The training protocol mirrors standard neural network training, but with an explicit "exchange" step that represents the network communication between the two parties:

Forward pass:

1. Publisher computes its 32-dimensional hidden representation from its local features.
2. Publisher sends h_local to the aggregator (hosted by the advertiser).
3. Advertiser computes its 128-dimensional hidden representation from its local features.
4. Aggregator concatenates both representations and produces P(CVR).

Backward pass:

1. Aggregator computes the loss using the conversion label (owned by the advertiser).
2. Aggregator backpropagates, computing gradients for both h_local and h_fed.
3. Gradient for h_local is sent back to the publisher over the network.

4. Each party runs local backpropagation on their own tower and updates their own parameters.

Our implementation simulates this entire protocol in a single process for pedagogical clarity. In a production deployment, the forward and backward communication would occur over a secure network, and each party's tower would run on their own infrastructure.

Training settings match the FedAds paper:

Setting	Value	Rationale
Embedding dimension	8 per feature	Paper specification (Sec 5.1.4)
Batch size	256	Paper specification
Epochs	1	Standard for large-scale ad systems to prevent overfitting
Loss	Standard BCE	Paper does not use focal loss for this task
Train/test split	Time-based (row-order proxy)	Paper uses last week as test set

9.5.8 Privacy Is Not Free

VFL is not a silver bullet for privacy. Even though raw features are never exchanged, the hidden representations and gradients that are shared can still leak information that can be figured out using advanced techniques. The FedAds paper specifically evaluates label inference attacks: a curious publisher could analyze the gradients it receives to infer whether specific samples were positive (converted) or negative. This is a real threat, not just theoretical.

The paper explores several defenses:

1. Differential Privacy (DP): Add calibrated Gaussian noise to the exchanged hidden representations or gradients. The paper finds that light noise ($\varepsilon = 10$) barely hurts prediction quality, while strong noise ($\varepsilon = 1$) causes a meaningful AUC drop.
2. MixPro: The paper's proposed defense that mixes gradients within a batch (making it harder to attribute any single gradient to a specific sample) and projects out directions that leak label information.
3. Gradient compression: Quantize gradients to low precision before sharing, reducing the information content available for attacks.

For production VFL deployments, the privacy story extends well beyond the model: it includes secure communication channels, access controls, contractual agreements between parties, and compliance with privacy regulations. The ML model is just one component of a broader governance framework.

9.6 Practical Considerations

9.6.1 Delayed Feedback

In advertising, conversions can occur hours or days after a click. A user clicks a sponsored product at 9am, browses other products, adds to cart at lunch, and finally purchases at 9pm. If the model trains on data collected at 3pm, that sample would be labeled "no conversion", a false negative. This is called delayed feedback, and it biases CVR estimates downward for recent data.

The simplest correction is importance weighting: recent negative samples are down-weighted proportionally to the fraction of the attribution window that has elapsed:

```
weight = min(1.0, time_since_click / attribution_window)
weight[converted == 1] = 1.0  # Observed conversions are certain
```

A sample observed at 50% of the attribution window gets 50% weight (under a linear assumption). Observed conversions always get full weight, because once you see a conversion, it is certain—the delay has been resolved. Only recent negatives are uncertain, because the user might still convert.

A common mistake is to discard all recent samples (both positives and negatives) until the full attribution window has passed. This is unnecessarily wasteful: observed conversions should always be kept. The code in utils/delayed_feedback.py implements both the importance weighting approach and an exponential variant (as a homework exercise) that better models the empirical observation that most conversions happen quickly after the click.

9.6.2 Evaluation Metrics for Advertising

We covered calibration (ECE) in Section 9.3.7. Here we briefly summarize the full set of evaluation metrics used in the accompanying code:

Metric	What It Measures	When to Use It
Global AUC	Overall ranking quality across all users	Quick sanity check, but can be misleading for personalization
GAUC (Grouped AUC)	Average per-user ranking quality	Primary metric for personalized ad ranking
Log Loss	Quality of predicted probabilities	When probability accuracy matters (i.e., always in ads)
ECE	Gap between predicted and actual probabilities	Directly relevant to auction pricing accuracy
Lift@k	How much better is top-k% vs. random targeting?	Business impact: "our top 10% converts 3.5× better than average"

Grouped AUC (GAUC) deserves special mention. The familiar way of computing (global) AUC pools all users together, comparing positive-negative pairs across different users. But in production, the model ranks candidates for one user at a time, it never compares Alice's

clicked ad against Bob's non-clicked ad. GAUC computes AUC separately for each user and takes a weighted average, which better reflects real per-user ranking quality. This is the standard evaluation metric at large ad platforms. Implementation is in utils/metrics.py.

9.7 Summary

This chapter extended the multi-stage recommendation pipeline into the advertising domain. The key differences from organic recommendation are economic, not just technical:

1. The ML model feeds into an auction. Predicted probabilities are multiplied with real dollar bids, so calibration matters as much as ranking quality (AUC). A well-ranked but poorly calibrated model distorts prices, wastes budgets, and erodes advertiser trust.
2. Selection bias is a first-order problem in CVR modeling. Training on clicked samples only creates a distribution mismatch between training and inference. ESMM solves this through the multiplicative decomposition CTCVR = CTR × CVR, enabling entire-space training on all impressions.
3. When data is siloed across organizations, Vertical Federated Learning enables joint modeling without sharing raw data. The split neural network architecture exchanges only hidden representations, preserving each party's data privacy at a modest cost to prediction quality.

The feedback loop between model, auction, and advertiser bidding creates a coupled system where model errors compound through economic incentives. Understanding this loop is essential for building advertising systems that are stable over time.

References

[1] Ma et al. "Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate" (SIGIR 2018)
[2] Wei et al. "FedAds: A Benchmark for Privacy-Preserving CVR Estimation with Vertical Federated Learning" (SIGIR 2023)
[3] Lin et al. "Focal Loss for Dense Object Detection" (ICCV 2017)
[4] Chapelle "Modeling Delayed Feedback in Display Advertising" (KDD 2014)
[5] Guo et al. "On Calibration of Modern Neural Networks" (ICML 2017)
[6] Amazon.com, Inc. "Q4 2025 Earnings Release" (February 2026). Advertising services revenue of $69B for full-year 2025.

Code Reference

```
chapter9_adtech/
├── models/
│   ├── esmm.py                # ESMM + CVRBaseline models
│   └── split_nn.py            # SplitNN (VanillaVFL) + baselines
├── data/
│   ├── ali_ccp_dataset.py     # Ali-CCP data loader
│   └── fedads_loader.py       # FedAds data loader
├── utils/
```

```
│   ├── calibration.py          # Platt, isotonic, temperature scaling + ECE
│   ├── delayed_feedback.py     # Importance weighting for delayed conversions
│   └── metrics.py              # GAUC, lift curves, popularity baselines
├── scripts/
│   └── preprocess_ali_ccp.py   # One-time Ali-CCP preprocessing
├── train_esmm.py               # ESMM training + baseline comparison
└── train_split_nn.py           # Federated learning training
```